

✓

Skin Health Analyzer — Enhanced (All 23 DermNet Classes + Flutter TFLite)

Developed By: Rijab Butt

Accuracy Enhancement Strategy

What Changed	Old Notebook	This Notebook
Classes	10 merged groups	23 original DermNet classes
Backbone	EfficientNetB0	EfficientNetB3 (higher capacity)
Augmentation	Basic (flip, rotate)	RandAugment + CutMix + ColorJitter
Class Imbalance	None handled	Weighted CrossEntropy loss
Normalization	ToTensor [0,1]	ImageNet mean/std normalization
Scheduler	None	CosineAnnealingLR
Fine-tuning	Full model	Gradual layer unfreezing
TFLite Export	Dynamic quant only	Float32 + Dynamic + INT8 (full integer)

✓

STEP 0 — Install Dependencies

```
!pip uninstall -y tensorflow tensorflow-cpu keras tf-keras protobuf onnx2tf
!pip cache purge

!pip install tensorflow==2.19.0
!pip install tf-keras==2.19.0
!pip install onnx onnxruntime
!pip install onnx2tf
```



```

Found existing installation: tensorflow 2.19.0
Uninstalling tensorflow-2.19.0:
  Successfully uninstalled tensorflow-2.19.0
WARNING: Skipping tensorflow-cpu as it is not installed.
Found existing installation: keras 3.14.1
Uninstalling keras-3.14.1:
  Successfully uninstalled keras-3.14.1
Found existing installation: tf_keras 2.19.0
Uninstalling tf_keras-2.19.0:
  Successfully uninstalled tf_keras-2.19.0
Found existing installation: protobuf 5.29.6
Uninstalling protobuf-5.29.6:
  Successfully uninstalled protobuf-5.29.6
WARNING: Skipping onnx2tf as it is not installed.
Files removed: 30
Collecting tensorflow==2.19.0
  Downloading tensorflow-2.19.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (592 bytes)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.19.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.19.0)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.19.0)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.19.0)
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.19.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.19.0)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.19.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.19.0)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<6.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.19.0)
  Downloading protobuf-5.29.6-cp38-abi3-manylinux2014_x86_64.whl.metadata (592 bytes)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from protobuf==5.29.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from protobuf==5.29.6)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from protobuf==5.29.6)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from protobuf==5.29.6)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from protobuf==5.29.6)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from protobuf==5.29.6)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from protobuf==5.29.6)
Requirement already satisfied: tensorboard~>2.19.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.19.0)
Collecting keras>=3.5.0 (from tensorflow==2.19.0)
  Downloading keras-3.14.1-py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: numpy<2.2.0,>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0)
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from rich==13.9.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from rich==13.9.4)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from rich==13.9.4)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from rich==13.9.4)
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from rich==13.9.4)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorboard==2.19.0)
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard==2.19.0)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug==3.0.1)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich==13.9.4)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich==13.9.4)
Requirement already satisfied: mdurl~>0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py==3.0.0)
  Downloading tensorflow-2.19.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (592.0 MB)

```

```

645.0/645.0 MB 1.4 MB/s eta 0:00:00
Downloading keras-3.14.1-py3-none-any.whl (1.6 MB)
1.6/1.6 MB 65.2 MB/s eta 0:00:00
Downloading protobuf-5.29.6-cp38-abi3-manylinux2014_x86_64.whl (320 kB)
320.5/320.5 kB 29.7 MB/s eta 0:00:
Installing collected packages: protobuf, keras, tensorflow
ERROR: pip's dependency resolver does not currently take into account all the
tensorflow-hub 0.16.1 requires tf-keras>=2.14.1, which is not installed.
dopamine-rl 4.1.2 requires tf-keras>=2.18.0, which is not installed.
ydf-tf 2.20.0 requires tensorflow==2.20.0, but you have tensorflow 2.19.0 which
tensorflow-text 2.20.1 requires tensorflow<2.21,>=2.20.0, but you have tensorf
Successfully installed keras-3.14.1 protobuf-5.29.6 tensorflow-2.19.0
Collecting tf-keras==2.19.0
  Downloading tf_keras-2.19.0-py3-none-any.whl.metadata (1.8 kB)
Requirement already satisfied: tensorflow<2.20,>=2.19 in /usr/local/lib/python
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/loc
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.1
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.1
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/pyth
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.1
Requirement already satisfied: tensorboard~=2.19.0 in /usr/local/lib/python3.1
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: numpy<2.2.0,>=1.26.0 in /usr/local/lib/python3.
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in /usr/local/lib/pytho
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/pyth
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/di
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/l
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/di
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/pytho
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-pa
Downloading tf_keras-2.19.0-py3-none-any.whl (1.7 MB)
1.7/1.7 MB 67.7 MB/s eta 0:00:00
Installing collected packages: tf-keras
Successfully installed tf-keras-2.19.0
Collecting onnx
  Downloading onnx-1.22.0-cp312-abi3-manylinux_2_27_x86_64_manylinux_2_28_x86

```

```

Collecting onnxruntime
  Downloading onnxruntime-1.27.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_
Requirement already satisfied: numpy>=1.23.2 in /usr/local/lib/python3.12/dist
Requirement already satisfied: protobuf>=4.25.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: typing_extensions>=4.15.0 in /usr/local/lib/pyt
Requirement already satisfied: ml_dtypes>=0.5.4 in /usr/local/lib/python3.12/d
Requirement already satisfied: flatbuffers in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-pac
Downloading onnx-1.22.0-cp312-abi3-manylinux_2_27_x86_64.manylinux_2_28_x86_64
----- 19.1/19.1 MB 88.8 MB/s eta 0:00:00
Downloading onnxruntime-1.27.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_2
----- 18.7/18.7 MB 97.4 MB/s eta 0:00:00
Installing collected packages: onnxruntime, onnx
Successfully installed onnx-1.22.0 onnxruntime-1.27.0
Collecting onnx2tf
  Downloading onnx2tf-2.4.2-py3-none-any.whl.metadata (223 kB)
----- 223.2/223.2 kB 15.1 MB/s eta 0:00:
Collecting numpy==1.26.4 (from onnx2tf)
  Downloading numpy-1.26.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_
----- 61.0/61.0 kB 6.4 MB/s eta 0:00:0
Collecting onnx==1.20.1 (from onnx2tf)
  Downloading onnx-1.20.1-cp312-abi3-manylinux_2_27_x86_64.manylinux_2_28_x86_
Collecting onnxruntime==1.24.3 (from onnx2tf)
  Downloading onnxruntime-1.24.3-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_
Collecting opencv-python==4.11.0.86 (from onnx2tf)
  Downloading opencv_python-4.11.0.86-cp37-abi3-manylinux_2_17_x86_64.manylinu
Collecting onnxsim-prebuilt==0.4.39.post2 (from onnx2tf)
  Downloading onnxsim_prebuilt-0.4.39.post2-cp312-abi3-manylinux_2_27_x86_64.m
Collecting onnxoptimizer==0.4.2 (from onnx2tf)
  Downloading onnxoptimizer-0.4.2-cp312-abi3-manylinux_2_24_x86_64.manylinux_2_
Collecting onnxscript==0.6.2 (from onnx2tf)
  Downloading onnxscript-0.6.2-py3-none-any.whl.metadata (13 kB)
Collecting ai-edge-litert==2.1.2 (from onnx2tf)
  Downloading ai_edge_litert-2.1.2-cp312-cp312-manylinux_2_27_x86_64.whl.metad
Collecting sne4onnx==2.0.1 (from onnx2tf)
  Downloading sne4onnx-2.0.1-py3-none-any.whl.metadata (7.1 kB)
Collecting sng4onnx==2.0.1 (from onnx2tf)
  Downloading sng4onnx-2.0.1-py3-none-any.whl.metadata (4.6 kB)
Requirement already satisfied: psutil==5.9.5 in /usr/local/lib/python3.12/dist
Collecting protobuf==4.25.5 (from onnx2tf)
  Downloading protobuf-4.25.5-cp37-abi3-manylinux2014_x86_64.whl.metadata (541
Collecting h5py==3.12.1 (from onnx2tf)
  Downloading h5py-3.12.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_
Collecting ml-dtypes==0.5.1 (from onnx2tf)
  Downloading ml_dtypes-0.5.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_
Requirement already satisfied: flatbuffers==25.12.19 in /usr/local/lib/python3
Collecting tqdm==4.67.1 (from onnx2tf)
  Downloading tqdm-4.67.1-py3-none-any.whl.metadata (57 kB)
----- 57.7/57.7 kB 6.2 MB/s eta 0:00:0
Collecting pytest==9.0.2 (from onnx2tf)
  Downloading pytest-9.0.2-py3-none-any.whl.metadata (7.6 kB)
Collecting backports.strenum (from ai-edge-litert==2.1.2->onnx2tf)
  Downloading backports_strenum-1.2.8-py3-none-any.whl.metadata (3.6 kB)
Requirement already satisfied: typing_extensions in /usr/local/lib/python3.12/
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: sympy in /usr/local/lib/python3.12/dist-package

```

```

Collecting onnx_ir<2,>=0.1.15 (from onnxscript==0.6.2->onnx2tf)
  Downloading onnx_ir-0.2.1-py3-none-any.whl.metadata (3.3 kB)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: iniconfig>=1.0.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: pluggy<2,>=1.5 in /usr/local/lib/python3.12/dis
Requirement already satisfied: pygments>=2.7.2 in /usr/local/lib/python3.12/di
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-pa
Downloading onnx2tf-2.4.2-py3-none-any.whl (1.9 MB)
  _____ 1.9/1.9 MB 91.6 MB/s eta 0:00:00
Downloading ai_edge_litert-2.1.2-cp312-cp312-manylinux_2_27_x86_64.whl (16.3 M
  _____ 16.3/16.3 MB 88.6 MB/s eta 0:00:00
Downloading h5py-3.12.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64
  _____ 5.4/5.4 MB 108.8 MB/s eta 0:00:00
Downloading ml_dtypes-0.5.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x8
  _____ 4.7/4.7 MB 81.7 MB/s eta 0:00:00
Downloading numpy-1.26.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_6
  _____ 18.0/18.0 MB 26.7 MB/s eta 0:00:00
Downloading onnx-1.20.1-cp312-abi3-manylinux_2_27_x86_64.manylinux_2_28_x86_64
  _____ 17.5/17.5 MB 81.4 MB/s eta 0:00:00
Downloading onnxoptimizer-0.4.2-cp312-abi3-manylinux_2_24_x86_64.manylinux_2_2
  _____ 1.0/1.0 MB 67.4 MB/s eta 0:00:00
Downloading onnxruntime-1.24.3-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_2
  _____ 17.2/17.2 MB 85.4 MB/s eta 0:00:00
Downloading onnxscript-0.6.2-py3-none-any.whl (689 kB)
  _____ 689.1/689.1 kB 56.6 MB/s eta 0:00:
Downloading onnxsim_prebuilt-0.4.39.post2-cp312-abi3-manylinux_2_27_x86_64.man
  _____ 2.8/2.8 MB 106.3 MB/s eta 0:00:00
Downloading opencv_python-4.11.0.86-cp37-abi3-manylinux_2_17_x86_64.manylinux2
  _____ 63.0/63.0 MB 10.7 MB/s eta 0:00:00
Downloading protobuf-4.25.5-cp37-abi3-manylinux2014_x86_64.whl (294 kB)
  _____ 294.6/294.6 kB 29.7 MB/s eta 0:00:
Downloading pytest-9.0.2-py3-none-any.whl (374 kB)
  _____ 374.8/374.8 kB 34.8 MB/s eta 0:00:
Downloading sne4onnx-2.0.1-py3-none-any.whl (12 kB)
Downloading sng4onnx-2.0.1-py3-none-any.whl (11 kB)
Downloading tqdm-4.67.1-py3-none-any.whl (78 kB)
  _____ 78.5/78.5 kB 9.4 MB/s eta 0:00:00
Downloading onnx_ir-0.2.1-py3-none-any.whl (166 kB)
  _____ 166.8/166.8 kB 17.6 MB/s eta 0:00:
Downloading backports_strenum-1.2.8-py3-none-any.whl (7.9 kB)
Installing collected packages: tqdm, sng4onnx, sne4onnx, pytest, protobuf, num
  Attempting uninstall: tqdm
    Found existing installation: tqdm 4.67.3
    Uninstalling tqdm-4.67.3:
      Successfully uninstalled tqdm-4.67.3
  Attempting uninstall: pytest
    Found existing installation: pytest 8.4.2
    Uninstalling pytest-8.4.2:
      Successfully uninstalled pytest-8.4.2
  Attempting uninstall: protobuf
    Found existing installation: protobuf 5.29.6
    Uninstalling protobuf-5.29.6:
      Successfully uninstalled protobuf-5.29.6
  Attempting uninstall: numpy
    Found existing installation: numpy 2.0.2

```



```

Found existing installation: numpy 2.0.2
Uninstalling numpy-2.0.2:
  Successfully uninstalled numpy-2.0.2
Attempting uninstall: opencv-python
  Found existing installation: opencv-python 4.13.0.92
  Uninstalling opencv-python-4.13.0.92:
    Successfully uninstalled opencv-python-4.13.0.92
Attempting uninstall: onnxruntime
  Found existing installation: onnxruntime 1.27.0
  Uninstalling onnxruntime-1.27.0:
    Successfully uninstalled onnxruntime-1.27.0
Attempting uninstall: ml-dtypes
  Found existing installation: ml_dtypes 0.5.4
  Uninstalling ml_dtypes-0.5.4:
    Successfully uninstalled ml_dtypes-0.5.4
Attempting uninstall: h5py
  Found existing installation: h5py 3.16.0
  Uninstalling h5py-3.16.0:
    Successfully uninstalled h5py-3.16.0
Attempting uninstall: onnx
  Found existing installation: onnx 1.22.0
  Uninstalling onnx-1.22.0:
    Successfully uninstalled onnx-1.22.0

```

ERROR: pip's dependency resolver does not currently take into account all the google-cloud-datastore 2.25.0 requires protobuf<8.0.0,>=4.25.8, but you have p grpc-google-iam-v1 0.14.4 requires protobuf<8.0.0,>=4.25.8, but you have proto google-cloud-dataplex 2.20.0 requires protobuf<8.0.0,>=4.25.8, but you have pr google-api-core 2.30.3 requires protobuf<8.0.0,>=4.25.8, but you have protobuf google-cloud-bigquery-storage 2.39.0 requires protobuf<8.0.0,>=4.25.8, but you xarray-einstats 0.10.0 requires numpy>=2.0, but you have numpy 1.26.4 which is opencv-contrib-python 4.13.0.92 requires numpy>=2; python_version >= "3.9", but pytensor 2.38.3 requires numpy>=2.0, but you have numpy 1.26.4 which is incom google-cloud-dataproc 5.28.0 requires protobuf<8.0.0,>=4.25.8, but you have pr grpcio-status 1.71.2 requires protobuf<6.0dev,>=5.26.1, but you have protobuf tiffiffle 2026.4.11 requires numpy>=2.0, but you have numpy 1.26.4 which is inc google-cloud-language 2.20.0 requires protobuf<8.0.0,>=4.25.8, but you have pr google-cloud-bigtable 2.38.0 requires protobuf<8.0.0,>=4.25.8, but you have pr cupy-cuda12x 14.0.1 requires numpy<2.6,>=2.0, but you have numpy 1.26.4 which shap 0.52.0 requires numpy>=2, but you have numpy 1.26.4 which is incompatible grain 0.2.17 requires protobuf>=5.28.3, but you have protobuf 4.25.5 which is opencv-python-headless 4.13.0.92 requires numpy>=2; python_version >= "3.9", b ydf-tf 2.20.0 requires tensorflow==2.20.0, but you have tensorflow 2.19.0 whic jax 0.7.2 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible tensorflow-text 2.20.1 requires tensorflow<2.21,>=2.20.0, but you have tensorf google-cloud-iam 2.23.0 requires protobuf<8.0.0,>=4.25.8, but you have protobu google-cloud-pubsub 2.39.0 requires protobuf<8.0.0,>=4.25.8, but you have prot tobler 0.14.0 requires numpy>=2.0, but you have numpy 1.26.4 which is incompat proto-plus 1.28.0 requires protobuf<8.0.0,>=4.25.8, but you have protobuf 4.25 google-cloud-secret-manager 2.29.0 requires protobuf<8.0.0,>=4.25.8, but you h googleapis-common-protos 1.75.0 requires protobuf<8.0.0,>=4.25.8, but you have google-cloud-bigquery-connection 1.22.0 requires protobuf<8.0.0,>=4.25.8, but : google-cloud-translate 3.26.0 requires protobuf<8.0.0,>=4.25.8, but you have p google-cloud-functions 1.23.0 requires protobuf<8.0.0,>=4.25.8, but you have p google-cloud-resource-manager 1.17.0 requires protobuf<8.0.0,>=4.25.8, but you google-cloud-logging 3.16.0 requires protobuf<8.0.0,>=4.25.8, but you have pro google-cloud-spanner 3.67.0 requires protobuf<8.0.0,>=4.25.8, but you have pro ydf 0.15.0 requires protobuf<7.0.0,>=5.29.1, but you have protobuf 4.25.5 whic

```
import tensorflow as tf
print(tf.__version__)
```

~~google-cloud-speech 2.40.0 requires protobuf<8.0.0,>=4.25.8, but you have protobu~~
~~google-cloud-audit-log 0.6.0 requires protobuf<8.0.0,>=4.25.8, but you have pr~~
~~google-cloud-monitoring 2.31.0 requires protobuf<8.0.0,>=4.25.8, but you have~~
~~google-cloud-firestore 2.27.0 requires protobuf<8.0.0,>=4.25.8, but you have p~~
~~opentelemetry-proto 1.35.0 requires protobuf<7.0.0,>=5.0, but you have protobuf~~
~~rasterio 1.5.0 requires numpy>=2, but you have numpy 1.26.4 which is incompati~~
 Successfully installed ai-edge-litert-2.1.2 backports.strenum-1.2.8 h5py-3.12.

STEP 1 - Download DermNet + HAM10000 Datasets

```
import os
from google.colab import files

files.upload() # Upload kaggle.json
os.makedirs('/root/.kaggle', exist_ok=True)
!cp kaggle.json /root/.kaggle/
!chmod 600 /root/.kaggle/kaggle.json

# DermNet (23 classes)
!kaggle datasets download -d shubhamgoel27/dermnet
!unzip -qo dermnet.zip -d dermnet

# HAM10000 (dermoscopy quality boost for Malignant/Benign classes)
!kaggle datasets download -d kmader/skin-cancer-mnist-ham10000
!unzip -qo skin-cancer-mnist-ham10000.zip -d ham10000

# Unzip nested HAM10000 image parts
for part in ['HAM10000_images_part1.zip', 'HAM10000_images_part2.zip']:
    p = os.path.join('ham10000', part)
    if os.path.exists(p):
        !unzip -qo {p} -d ham10000
        print(f'Unzipped {part}')
```

Choose files kaggle.json

kaggle.json(application/json) - 66 bytes, last modified: 03/04/2026 - 100% done

Saving kaggle.json to kaggle.json

Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2)

Dataset URL: <https://www.kaggle.com/datasets/shubhamgoel27/dermnet>

License(s): Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-

Downloading dermnet.zip to /content

100% 1.72G/1.72G [01:37<00:00, 19.0MB/s]

Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2)

Dataset URL: <https://www.kaggle.com/datasets/kmader/skin-cancer-mnist-ham10000>

License(s): CC-BY-NC-SA-4.0

Downloading skin-cancer-mnist-ham10000.zip to /content

100% 5.20G/5.20G [04:56<00:00, 18.9MB/s]

STEP 2 — Discover All 23 DermNet Classes

```
import os
from collections import Counter

DERMNET_PATH = 'dermnet'

# Discover all category folders from both train and test splits
all_categories = set()
for split in ['train', 'test']:
    split_path = os.path.join(DERMNET_PATH, split)
    if os.path.isdir(split_path):
        for cat in os.listdir(split_path):
            if os.path.isdir(os.path.join(split_path, cat)):
                all_categories.add(cat)

# Sort and display all discovered categories
CLASS_NAMES = sorted(list(all_categories))
NUM_CLASSES = len(CLASS_NAMES)
print(f'Total DermNet classes found: {NUM_CLASSES}')
for i, c in enumerate(CLASS_NAMES):
    print(f' {i:02d}: {c}')

# Build class-to-index mapping
CLASS_TO_IDX = {c: i for i, c in enumerate(CLASS_NAMES)}
```

```
Total DermNet classes found: 23
00: Acne and Rosacea Photos
01: Actinic Keratosis Basal Cell Carcinoma and other Malignant Lesions
02: Atopic Dermatitis Photos
03: Bullous Disease Photos
04: Cellulitis Impetigo and other Bacterial Infections
05: Eczema Photos
06: Exanthems and Drug Eruptions
07: Hair Loss Photos Alopecia and other Hair Diseases
08: Herpes HPV and other STDs Photos
09: Light Diseases and Disorders of Pigmentation
10: Lupus and other Connective Tissue diseases
11: Melanoma Skin Cancer Nevi and Moles
12: Nail Fungus and other Nail Disease
13: Poison Ivy Photos and other Contact Dermatitis
14: Psoriasis pictures Lichen Planus and related diseases
15: Scabies Lyme Disease and other Infestations and Bites
16: Seborrheic Keratoses and other Benign Tumors
17: Systemic Disease
18: Tinea Ringworm Candidiasis and other Fungal Infections
19: Urticaria Hives
20: Vascular Tumors
21: Vasculitis Photos
22: Warts Molluscum and other Viral Infections
```

✓ STEP 3 — Build Data Directory (All 23 Classes + HAM10000 Augmentation)

```

import shutil
import pandas as pd
from sklearn.model_selection import train_test_split

BASE_DATA_DIR = 'data'
TRAIN_DIR = os.path.join(BASE_DATA_DIR, 'train')
VAL_DIR = os.path.join(BASE_DATA_DIR, 'val')

# Create class directories for all 23 classes
for cls in CLASS_NAMES:
    os.makedirs(os.path.join(TRAIN_DIR, cls), exist_ok=True)
    os.makedirs(os.path.join(VAL_DIR, cls), exist_ok=True)

print('Created directory structure for', NUM_CLASSES, 'classes')

# — Collect all DermNet images (combine train + test, then re-split) —
all_imgs, all_labels = [], []

for split in ['train', 'test']:
    split_path = os.path.join(DERMNET_PATH, split)
    if not os.path.isdir(split_path):
        continue
    for cat in os.listdir(split_path):
        cat_path = os.path.join(split_path, cat)
        if not os.path.isdir(cat_path) or cat not in CLASS_TO_IDX:
            continue
        for fname in os.listdir(cat_path):
            if fname.lower().endswith(('.jpg', '.jpeg', '.png')):
                all_imgs.append(os.path.join(cat_path, fname))
                all_labels.append(cat)

print(f'Total DermNet images collected: {len(all_imgs)}')
print('Per-class counts:', Counter(all_labels))

# Stratified 80/20 split
train_imgs, val_imgs, train_labels, val_labels = train_test_split(
    all_imgs, all_labels, test_size=0.2, stratify=all_labels, random_state=42
)

def copy_images(img_paths, labels, dest_root):
    for src, lbl in zip(img_paths, labels):
        dst = os.path.join(dest_root, lbl, os.path.basename(src))
        # Handle name collisions by appending index
        if os.path.exists(dst):

```

```

        base, ext = os.path.splitext(os.path.basename(src))
        dst = os.path.join(dest_root, lbl, f'{base}_{id(src)}{ext}')
        shutil.copy(src, dst)

copy_images(train_imgs, train_labels, TRAIN_DIR)
copy_images(val_imgs, val_labels, VAL_DIR)
print('DermNet images copied.')

# — Add HAM10000 to boost Melanoma Skin Cancer Nevi and Moles class —
# HAM10000 'mel' (melanoma) + 'nv' (nevi) map to DermNet's melanoma/nevi category
HAM_CLASS_TARGET = None
for cls in CLASS_NAMES:
    if 'melanoma' in cls.lower() or 'nevi' in cls.lower():
        HAM_CLASS_TARGET = cls
        break

if HAM_CLASS_TARGET:
    print(f'Mapping HAM10000 mel/nv images to: "{HAM_CLASS_TARGET}"')
    HAM_META = 'ham10000/HAM10000_metadata.csv'
    HAM_DIRS = ['ham10000/HAM10000_images_part_1', 'ham10000/HAM10000_images_part_2']

    if os.path.exists(HAM_META):
        df = pd.read_csv(HAM_META)
        mel_nv_ids = set(df[df['dx'].isin(['mel', 'nv'])]['image_id'].tolist())
        ham_imgs, ham_labels = [], []
        for hdir in HAM_DIRS:
            if not os.path.isdir(hdir):
                continue
            for fname in os.listdir(hdir):
                img_id = os.path.splitext(fname)[0]
                if img_id in mel_nv_ids and fname.lower().endswith(('.jpg', '.jpeg')):
                    ham_imgs.append(os.path.join(hdir, fname))
                    ham_labels.append(HAM_CLASS_TARGET)
        if ham_imgs:
            h_train, h_val, hl_train, hl_val = train_test_split(
                ham_imgs, ham_labels, test_size=0.2, stratify=ham_labels, random_state=42)
            copy_images(h_train, hl_train, TRAIN_DIR)
            copy_images(h_val, hl_val, VAL_DIR)
            print(f'HAM10000: added {len(h_train)} train + {len(h_val)} val images')
        else:
            print('No melanoma/nevi class found in DermNet – skipping HAM10000 injection')

# — Final class distribution check —
print('\nFinal training set distribution:')
for cls in CLASS_NAMES:
    n = len(os.listdir(os.path.join(TRAIN_DIR, cls)))
    print(f' {cls}: {n}')

```

Created directory structure for 23 classes
 Total DermNet images collected: 19559

```

Per-class counts: Counter({'Psoriasis pictures Lichen Planus and related disease': 1405, 'DermNet images copied': 1, 'Melanoma Skin Cancer Nevus and Moles': 6717})
Mapping HAM10000 mel/nv images to: "Melanoma Skin Cancer Nevus and Moles"
HAM10000: added 6254 train + 1564 val images for "Melanoma Skin Cancer Nevus and Moles"

Final training set distribution:
Acne and Rosacea Photos: 922
Actinic Keratosis Basal Cell Carcinoma and other Malignant Lesions: 1150
Atopic Dermatitis Photos: 490
Bullous Disease Photos: 449
Cellulitis Impetigo and other Bacterial Infections: 289
Eczema Photos: 1235
Exanthems and Drug Eruptions: 404
Hair Loss Photos Alopecia and other Hair Diseases: 239
Herpes HPV and other STDs Photos: 406
Light Diseases and Disorders of Pigmentation: 569
Lupus and other Connective Tissue diseases: 420
Melanoma Skin Cancer Nevus and Moles: 6717
Nail Fungus and other Nail Disease: 1041
Poison Ivy Photos and other Contact Dermatitis: 260
Psoriasis pictures Lichen Planus and related diseases: 1405
Scabies Lyme Disease and other Infestations and Bites: 431
Seborrheic Keratoses and other Benign Tumors: 1371
Systemic Disease: 606
Tinea Ringworm Candidiasis and other Fungal Infections: 1300
Urticaria Hives: 212
Vascular Tumors: 482
Vasculitis Photos: 417
Warts Molluscum and other Viral Infections: 1086

```

✓ STEP 4 — Dataset Loader with Strong Augmentation

```

import torch
import torch.nn as nn
import numpy as np
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, WeightedRandomSampler

IMG_SIZE = 300 # EfficientNetB3 native resolution
BATCH_SIZE = 32

# ImageNet mean/std — critical for pretrained EfficientNet
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]

# — Training transforms: strong augmentation —
train_tfms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.RandomRotation(15),
])

```

```

    transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.3, hue=0.
    transforms.RandomAffine(degrees=0, translate=(0.1, 0.1), scale=(0.9, 1.1)),
    # RandAugment for further diversity
    transforms.RandAugment(num_ops=2, magnitude=9),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
    transforms.RandomErasing(p=0.2), # CutOut / random erasing
])

# — Validation transforms: no augmentation —
val_tfms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
])

train_data = datasets.ImageFolder(TRAIN_DIR, transform=train_tfms)
val_data = datasets.ImageFolder(VAL_DIR, transform=val_tfms)

NUM_CLASSES = len(train_data.classes)
CLASS_NAMES = train_data.classes
print(f'Classes ({NUM_CLASSES}):', CLASS_NAMES)

# — Weighted sampler to handle class imbalance —
# Count samples per class
class_counts = np.array([len(os.listdir(os.path.join(TRAIN_DIR, c))) for c in C
class_weights = 1.0 / class_counts
sample_weights = np.array([class_weights[train_data.targets[i]] for i in range(
sampler = WeightedRandomSampler(
    weights=torch.from_numpy(sample_weights).float(),
    num_samples=len(train_data),
    replacement=True
)

train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, sampler=sampler,
val_loader = DataLoader(val_data, batch_size=BATCH_SIZE, shuffle=False, num

print(f'Train batches: {len(train_loader)} | Val batches: {len(val_loader)}')

Classes (23): ['Acne and Rosacea Photos', 'Actinic Keratosis Basal Cell Carcinom
Train batches: 685 | Val batches: 172

```

✓ STEP 5 — Model: EfficientNetB3 with Custom Head

```

from torchvision import models

# EfficientNetB3 — better accuracy than B0, still mobile-exportable
# B0 params: 5.3M | B3 params: 12M | Size tradeoff worth it for 23-class task

```

```

backbone = models.efficientnet_b3(weights='IMAGENET1K_V1')

# — Phase 1: Freeze all backbone layers —
for param in backbone.parameters():
    param.requires_grad = False

# Replace classifier head
in_features = backbone.classifier[1].in_features # 1536 for B3
backbone.classifier = nn.Sequential(
    nn.Dropout(p=0.4),
    nn.Linear(in_features, 512),
    nn.SiLU(), # Swish — same activation as EfficientNet
    nn.BatchNorm1d(512),
    nn.Dropout(p=0.3),
    nn.Linear(512, NUM_CLASSES),
)

model = backbone.cuda()

# Count trainable params
total = sum(p.numel() for p in model.parameters())
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f'Total params: {total/1e6:.2f}M | Trainable (head only): {trainable/1e6:.2f}M')

```

Total params: 11.50M | Trainable (head only): 0.80M

✓ STEP 6 — Loss Function with Class Weighting

```

import torch

# Compute inverse-frequency class weights for CrossEntropy
class_counts_tensor = torch.tensor(class_counts, dtype=torch.float32)
ce_class_weights = (1.0 / class_counts_tensor)
ce_class_weights = ce_class_weights / ce_class_weights.sum() * NUM_CLASSES # N
ce_class_weights = ce_class_weights.cuda()

criterion = nn.CrossEntropyLoss(weight=ce_class_weights, label_smoothing=0.1)
print('Weighted CrossEntropyLoss with label_smoothing=0.1 ready.')
print('Class weights (top 5 highest):')
top5 = ce_class_weights.cpu().argsort(descending=True)[:5]
for idx in top5:
    print(f' {CLASS_NAMES[idx]}: {ce_class_weights[idx]:.4f}')

```

Weighted CrossEntropyLoss with label_smoothing=0.1 ready.
Class weights (top 5 highest):
Urticaria Hives: 2.4109
Hair Loss Photos Alopecia and other Hair Diseases: 2.1385
Poison Ivy Photos and other Contact Dermatitis: 1.9658
Cellulitis Impetigo and other Bacterial Infections: 1.7686
Exanthems and Drug Eruptions: 1.2651

✓ STEP 7 — Evaluation Helper

```
from sklearn.metrics import f1_score, classification_report

def evaluate(model, loader, verbose=False):
    model.eval()
    preds, targets = [], []
    with torch.no_grad():
        for x, y in loader:
            x, y = x.cuda(), y.cuda()
            out = model(x)
            p = out.argmax(1)
            preds.extend(p.cpu().numpy())
            targets.extend(y.cpu().numpy())
    f1 = f1_score(targets, preds, average='weighted')
    if verbose:
        print(classification_report(targets, preds, target_names=CLASS_NAMES, z
    return f1
```

✓ STEP 8 — Phase 1: Train Head Only (7 Epochs)

```
import torch.optim as optim

# Only classifier head is trainable at this point
optimizer = optim.AdamW(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=1e-3, weight_decay=1e-4
)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=5, eta_min=1e-6)

print('Phase 1: Training classifier head only...')
for epoch in range(7):
    model.train()
    running_loss = 0.0
    for x, y in train_loader:
        x, y = x.cuda(), y.cuda()
        optimizer.zero_grad()
        loss = criterion(model(x), y)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    scheduler.step()
    f1 = evaluate(model, val_loader)
    avg_loss = running_loss / len(train_loader)
    print(f' Epoch {epoch+1}/7 | Loss: {avg_loss:.4f} | Val F1: {f1:.4f} | LR: {scheduler.get_lr()[0]:.4f}')
```



```
print('Phase 1 complete.')
```

```
Phase 1: Training classifier head only...
```

```
Epoch 1/7 | Loss: 2.4981 | Val F1: 0.3889 | LR: 0.000905
```

```
Epoch 2/7 | Loss: 2.4228 | Val F1: 0.3764 | LR: 0.000658
```

```
Epoch 3/7 | Loss: 2.3610 | Val F1: 0.3718 | LR: 0.000352
```

```
Epoch 4/7 | Loss: 2.3408 | Val F1: 0.4160 | LR: 0.000105
```

```
Epoch 5/7 | Loss: 2.3156 | Val F1: 0.4068 | LR: 0.000010
```

```
Epoch 6/7 | Loss: 2.3025 | Val F1: 0.3959 | LR: 0.000105
```

```
Epoch 7/7 | Loss: 2.2898 | Val F1: 0.4145 | LR: 0.000352
```

```
Phase 1 complete.
```

STEP 9 — Phase 2: Gradual Unfreezing — Unfreeze Last 2 Blocks (15 Epochs)

```
# Unfreeze the last 2 MBConv blocks of EfficientNetB3 features
# B3 features[0..7]: blocks 6 and 7 are the deepest semantic layers
for name, param in model.named_parameters():
    if 'features.6' in name or 'features.7' in name or 'classifier' in name:
        param.requires_grad = True

trainable2 = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f'Trainable params after unfreezing last 2 blocks: {trainable2/1e6:.2f}M')

optimizer2 = optim.AdamW(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=3e-4, weight_decay=1e-4
)
scheduler2 = optim.lr_scheduler.CosineAnnealingLR(optimizer2, T_max=10, eta_min=1e-5)

print('Phase 2: Fine-tuning last 2 blocks + head...')
best_f1 = 0.0
for epoch in range(15):
    model.train()
    running_loss = 0.0
    for x, y in train_loader:
        x, y = x.cuda(), y.cuda()
        optimizer2.zero_grad()
        loss = criterion(model(x), y)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer2.step()
        running_loss += loss.item()
    scheduler2.step()
    f1 = evaluate(model, val_loader)
    avg_loss = running_loss / len(train_loader)
    print(f' Epoch {epoch+1}/15 | Loss: {avg_loss:.4f} | Val F1: {f1:.4f}')
    if f1 > best_f1:
```

```

best_f1 = f1
# Ensure the path is consistent with loading and confirm save
save_path = '/content/best_skin_model_phase2.pt'
torch.save(model.state_dict(), save_path)
print(f'    ✓ Best model saved to {save_path} (F1={best_f1:.4f})')

print(f'Phase 2 complete. Best Val F1: {best_f1:.4f}')
files.download('/content/best_skin_model_phase2.pt')

```

```

Trainable params after unfreezing last 2 blocks: 8.71M
Phase 2: Fine-tuning last 2 blocks + head...
Epoch 1/15 | Loss: 2.1169 | Val F1: 0.4750
    ✓ Best model saved to /content/best_skin_model_phase2.pt (F1=0.4750)
Epoch 2/15 | Loss: 1.8848 | Val F1: 0.4886
    ✓ Best model saved to /content/best_skin_model_phase2.pt (F1=0.4886)

```

✓ STEP 10 — Phase 3: Full Fine-Tuning with Low LR (25 Epochs)

```

import torch.optim as optim
# Unfreeze ALL layers for final fine-tuning
for param in model.parameters():
    param.requires_grad = True

# Use differential learning rates: lower LR for backbone, higher for head
optimizer3 = optim.AdamW([
    {'params': model.features.parameters(), 'lr': 1e-5},
    {'params': model.classifier.parameters(), 'lr': 1e-4},
], weight_decay=1e-4)

scheduler3 = optim.lr_scheduler.CosineAnnealingLR(optimizer3, 25)

# Load best checkpoint from phase 2
model.load_state_dict(torch.load('/content/best_skin_model_phase2.pt'))
best_f1_phase3 = 0.6224

print('Phase 3: Full model fine-tuning...')
for epoch in range(25):
    model.train()
    running_loss = 0.0
    for x, y in train_loader:
        x, y = x.cuda(), y.cuda()
        optimizer3.zero_grad()
        loss = criterion(model(x), y)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer3.step()
        running_loss += loss.item()
    scheduler3.step()
    f1 = evaluate(model, val_loader)
    avg_loss = running_loss / len(train_loader)

```

```

print(f' Epoch {epoch+1}/25 | Loss: {avg_loss:.4f} | Val F1: {f1:.4f}')
if f1 > best_f1_phase3:
    best_f1_phase3 = f1
    torch.save(model.state_dict(), '/content/best_skin_model.pt')
    print(f'      ✓ Best model saved (F1={best_f1_phase3:.4f})')

print(f'Phase 3 complete. Best Val F1: {best_f1_phase3:.4f}')
files.download('/content/best_skin_model_final.pt')

```

```

Phase 3: Full model fine-tuning...
Epoch 1/35 | Loss: 1.2981 | Val F1: 0.7017
      ✓ Best model saved (F1=0.7017)
Epoch 2/35 | Loss: 1.2608 | Val F1: 0.6965
Epoch 3/35 | Loss: 1.2321 | Val F1: 0.6974
Epoch 4/35 | Loss: 1.1964 | Val F1: 0.7035
      ✓ Best model saved (F1=0.7035)
Epoch 5/35 | Loss: 1.1941 | Val F1: 0.7143
      ✓ Best model saved (F1=0.7143)
Epoch 6/35 | Loss: 1.1779 | Val F1: 0.7081
Epoch 7/35 | Loss: 1.1651 | Val F1: 0.7093
Epoch 8/35 | Loss: 1.1557 | Val F1: 0.7144
      ✓ Best model saved (F1=0.7144)
Epoch 9/35 | Loss: 1.1550 | Val F1: 0.7146
      ✓ Best model saved (F1=0.7146)
Epoch 10/35 | Loss: 1.1374 | Val F1: 0.7181
      ✓ Best model saved (F1=0.7181)
Epoch 11/35 | Loss: 1.1305 | Val F1: 0.7094
Epoch 12/35 | Loss: 1.1312 | Val F1: 0.7167
Epoch 13/35 | Loss: 1.1254 | Val F1: 0.7213
      ✓ Best model saved (F1=0.7213)
Epoch 14/35 | Loss: 1.1311 | Val F1: 0.7183
Epoch 15/35 | Loss: 1.1153 | Val F1: 0.7218
      ✓ Best model saved (F1=0.7218)
Epoch 16/35 | Loss: 1.1309 | Val F1: 0.7109
Epoch 17/35 | Loss: 1.1163 | Val F1: 0.7138
Epoch 18/35 | Loss: 1.1221 | Val F1: 0.7209
Epoch 19/35 | Loss: 1.1235 | Val F1: 0.7175
Epoch 20/35 | Loss: 1.1222 | Val F1: 0.7174
Phase 3 complete. Best Val F1: 0.7218

```

✓ STEP 11 — Final Evaluation with Per-Class Report

```

# Load best model
model.load_state_dict(torch.load('/content/best_skin_model_final.pt'))
print('Loaded best model. Running full evaluation...')
final_f1 = evaluate(model, val_loader, verbose=True)
print(f'\nFinal Weighted F1 Score: {final_f1:.4f}')

# Save class names for Flutter
import json
with open('labels.json', 'w') as f:

```

```

    json.dump(CLASS_NAMES, f, indent=2)
    print('Class labels saved to labels.json')

```

Loaded best model. Running full evaluation...

	precision
Acne and Rosacea Photos	0.78
Actinic Keratosis Basal Cell Carcinoma and other Malignant Lesions	0.68
Atopic Dermatitis Photos	0.61
Bullous Disease Photos	0.58
Cellulitis Impetigo and other Bacterial Infections	0.48
Eczema Photos	0.77
Exanthems and Drug Eruptions	0.55
Hair Loss Photos Alopecia and other Hair Diseases	0.43
Herpes HPV and other STDs Photos	0.49
Light Diseases and Disorders of Pigmentation	0.56
Lupus and other Connective Tissue diseases	0.57
Melanoma Skin Cancer Nevi and Moles	1.00
Nail Fungus and other Nail Disease	0.89
Poison Ivy Photos and other Contact Dermatitis	0.59
Psoriasis pictures Lichen Planus and related diseases	0.76
Scabies Lyme Disease and other Infestations and Bites	0.55
Seborrheic Keratoses and other Benign Tumors	0.71
Systemic Disease	0.61
Tinea Ringworm Candidiasis and other Fungal Infections	0.78
Urticaria Hives	0.53
Vascular Tumors	0.51
Vasculitis Photos	0.46
Warts Molluscum and other Viral Infections	0.72
accuracy	
macro avg	0.63
weighted avg	0.78

Final Weighted F1 Score: 0.7218
Class labels saved to labels.json

STEP 12 — Export: PyTorch → ONNX → TF SavedModel → TFLite

```

import torch
import torch.nn as nn
from torchvision import models
import onnx2tf
import shutil, os
import tensorflow as tf # Added tensorflow import
import traceback # Import traceback for detailed error logging
from google.colab import files # Import files for downloading

```

```

saved_model_base_path= '/content/tf_model'
float16_tflite_path='/content/tf_model/model_float16.tflite'
# — 4. Dynamic Range Quantization (using Float16 TFLite directly from onnx2tf)
dr_quant_tflite_path = os.path.join(saved_model_base_path, 'model_dr_quant.tfli

try:
    if os.path.exists(float16_tflite_path):
        shutil.copy(float16_tflite_path, dr_quant_tflite_path)
        print(f'✅ Dynamic Range Quant (Float16) TFLite: {os.path.getsize(dr_q
    else:
        raise FileNotFoundError(f"'model_float16.tflite' not found for Dynamic
except Exception as e:
    print(f'Dynamic Range Quantization failed: {e}')
    print(traceback.format_exc())
finally:
    if os.path.exists(dr_quant_tflite_path):
        print(f'File check: {dr_quant_tflite_path} exists and has size {os.pat
    else:
        print(f'File check: {dr_quant_tflite_path} DOES NOT exist.')
files.download('/content/tf_model/model_dr_quant.tflite')
# — 5. Full INT8 Quantization (currently challenging with this setup) —
# The current onnx2tf (2.4.2) version's quantize_int8 argument requires `repres
# (a list of directories with images) which is not directly compatible with the
# generator used for tf.lite.TFLiteConverter.from_saved_model().
# Re-quantizing an existing TFLite file (model_float32.tflite) to INT8 is not c
# tf.lite.TFLiteConverter.from_tflite_model() or from_buffer() in TensorFlow 2.
# Therefore, this step is skipped for now.

# int8_tflite_path = os.path.join(saved_model_base_path, 'model_int8.tflite')
# print('\nSkipping Full INT8 Quantization due to tool/version limitations.')
# try:
#     # Placeholder if an alternative method becomes available or onnx2tf is up
#     # For now, this part is commented out.
#     pass
# except Exception as e:
#     print(f'INT8 conversion failed (use dr_quant instead): {e}')
#     print(traceback.format_exc())
# finally:
#     if os.path.exists(int8_tflite_path):
#         print(f'File check: {int8_tflite_path} exists and has size {os.path.g
#     else:
#         print(f'File check: {int8_tflite_path} DOES NOT exist.')

print('\nAll available TFLite models exported!')

```

✅ Dynamic Range Quant (Float16) TFLite: 22.99 MB → /content/tf_model/model_dr_
 File check: /content/tf_model/model_dr_quant.tflite exists and has size 22992248

All available TFLite models exported!

STEP 13 — Verify TFLite Inference + Input/Output Shape Check

```

import numpy as np
import tensorflow as tf
import json
import os
import tensorflow as tf # Added tensorflow import
TFLITE_MODEL = '/content/tf_model/model_float32.tflite' # Use Float32 model file

interpreter = tf.lite.Interpreter(model_path=TFLITE_MODEL)
interpreter.allocate_tensors()

input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()

print('— Input Details —')
print(f' Shape: {input_details[0]["shape"]}') # Expected: [1, 3, 300, 300] channels
print(f' Dtype: {input_details[0]["dtype"]}')
print(f' Name: {input_details[0]["name"]}')

print('— Output Details —')
print(f' Shape: {output_details[0]["shape"]}') # Expected: [1, 23]
print(f' Dtype: {output_details[0]["dtype"]}')

# — Run dummy inference —
in_shape = input_details[0]['shape'] # (1, C, H, W) or (1, H, W, C)
in_dtype = input_details[0]['dtype']
dummy_input = np.random.rand(*in_shape).astype(in_dtype) # Cast to the model's dtype

interpreter.set_tensor(input_details[0]['index'], dummy_input)
interpreter.invoke()
output = interpreter.get_tensor(output_details[0]['index'])

# Softmax
def softmax(x):
    e = np.exp(x - np.max(x))
    return e / e.sum()

probs = softmax(output[0])
pred_idx = np.argmax(probs)

with open('labels.json') as f:
    labels = json.load(f)

print(f'\nDummy inference OK!')
print(f'Predicted class [{pred_idx}]: {labels[pred_idx]}')
print(f'Confidence: {probs[pred_idx]*100:.1f}%')

```

```
print(f'\nTop-3 predictions:')
top3 = np.argsort(probs)[::-1][:3]
for i, idx in enumerate(top3):
    print(f' {i+1}. {labels[idx]}: {probs[idx]*100:.1f}%')
```

— Input Details —

```
Shape: [ 1 300 300  3]
Dtype: <class 'numpy.float32'>
Name: input
```

— Output Details —

```
Shape: [ 1 23]
Dtype: <class 'numpy.float32'>
```

Dummy inference OK!

Predicted class [19]: Urticaria Hives

Confidence: 16.5%

Top-3 predictions:

1. Urticaria Hives: 16.5%
2. Poison Ivy Photos and other Contact Dermatitis: 14.0%
3. Cellulitis Impetigo and other Bacterial Infections: 7.3%

/usr/local/lib/python3.12/dist-packages/tensorflow/lite/python/interpreter.py:45
TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
See the [migration guide](<https://ai.google.dev/edge/litert/migration>)
for details.

```
warnings.warn(_INTERPRETER_DELETION_WARNING)
```

✓ STEP 14 — Save All Artifacts for Flutter

```
import zipfile

artifacts = [
    'model_dr_quant.tflite',    # Primary Flutter model (recommended)
    'model_float32.tflite',    # Fallback full-precision model
    'model_float16.tflite',    # Fallback dynamic-range model
    'labels.json',             # Class names in correct order
]

# Try INT8 if it was created
if os.path.exists('model_int8.tflite'):
    artifacts.append('model_int8.tflite')

with zipfile.ZipFile('flutter_model_assets.zip', 'w', zipfile.ZIP_DEFLATED) as
    for f in artifacts:
        if os.path.exists(f):
            zf.write(f)
            print(f' Added: {f} ({os.path.getsize(f)/1e6:.2f} MB)')

print('\n✅ flutter_model_assets.zip created!')
```

```
print('Place contents in your Flutter project under: assets/ml/')

# Download
from google.colab import files
files.download('flutter_model_assets.zip')
```

✓ STEP 15 — 6-Image Input Testing + Full Evaluation Stats

```
import os, random, json, warnings
import numpy as np
import torch
import torch.nn.functional as F
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
from PIL import Image
from torchvision import transforms
from sklearn.metrics import (
    accuracy_score, f1_score, precision_score,
    recall_score, confusion_matrix, classification_report
)
warnings.filterwarnings('ignore')

# — Colour palette —————
C_GREEN  = '#27ae60'
C_ORANGE = '#f39c12'
C_RED    = '#e74c3c'
C_BLUE   = '#2980b9'
C_PURPLE = '#8e44ad'
C_DARK   = '#2c3e50'

# —————
# 0. CONFIG
# —————
CONFIDENCE_THRESHOLD = 0.50      # below → "No Skin Issue Detected"
N_SAMPLE_IMAGES      = 6         # images to show in the panel

# Inference transform (same as val_tfms)
infer_tfm = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
])

# Load best model
model.load_state_dict(torch.load('/content/best_skin_model_phase2.pt'))
model.eval()
```



```

model.cuda()
print("Model loaded: best_skin_model_phase2.pt")

# -----
# 1. HELPER FUNCTIONS
# -----
def softmax_np(logits):
    e = np.exp(logits - logits.max())
    return e / e.sum()

def predict_single(img_pil):
    """Returns (probs, pred_idx, pred_label, confidence, ood_flag)"""
    tensor = infer_tfm(img_pil).unsqueeze(0).cuda()
    with torch.no_grad():
        logits = model(tensor)
        probs = F.softmax(logits, dim=1).cpu().numpy()[0]
    pred_idx = probs.argmax()
    confidence = probs.max()
    entropy = -(probs * np.log(probs + 1e-9)).sum()
    entropy_thr = np.log(NUM_CLASSES) * 0.85 # 85% of max possible entropy
    if confidence < CONFIDENCE_THRESHOLD or entropy > entropy_thr:
        pred_label = "No Skin Issue Detected"
        ood_flag = True
    else:
        pred_label = CLASS_NAMES[pred_idx]
        ood_flag = False
    return probs, int(pred_idx), pred_label, float(confidence), ood_flag

def denorm_display(tensor):
    """Denormalize ImageNet tensor → (H,W,3) numpy clipped to [0,1]"""
    d = tensor.clone()
    for ch, m, s in zip(range(3), IMAGENET_MEAN, IMAGENET_STD):
        d[ch] = d[ch] * s + m
    return d.permute(1, 2, 0).clamp(0, 1).numpy()

def short(cls_name, maxlen=28):
    s = cls_name.replace(' Photos','').replace(' and other',' &')
    return s[:maxlen]

# -----
# 2. PICK 5 SAMPLE IMAGES (one per unique class if possible)
# -----

sample_imgs_pil = []
sample_true_lbls = [] # ground-truth class name (or 'uploaded' if custom)
sample_paths = []

CUSTOM_DIR = '/content/test_images'
if os.path.isdir(CUSTOM_DIR):
    exts = ('.jpg', '.jpeg', '.png')
    custom_files = [os.path.join(CUSTOM_DIR, f)
                     for f in sorted(os.listdir(CUSTOM_DIR))

```

```

        if f.lower().endswith(EXTS)][0:N_SAMPLE_IMAGES]
    for p in custom_files:
        sample_imgs_pil.append(Image.open(p).convert('RGB'))
        sample_true_lbls.append('uploaded')
        sample_paths.append(p)
    print(f"Using {len(sample_imgs_pil)} custom images from {CUSTOM_DIR}")
else:
    # Auto-pick: one random image from N_SAMPLE_IMAGES different classes
    chosen_classes = random.sample(CLASS_NAMES, min(N_SAMPLE_IMAGES, NUM_CLASSES))
    for cls in chosen_classes:
        cls_dir = os.path.join(VAL_DIR, cls)
        if not os.path.isdir(cls_dir):
            continue
        imgs = [f for f in os.listdir(cls_dir)
                 if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
        if not imgs:
            continue
        p = os.path.join(cls_dir, random.choice(imgs))
        sample_imgs_pil.append(Image.open(p).convert('RGB'))
        sample_true_lbls.append(cls)
        sample_paths.append(p)
    print(f"Auto-picked {len(sample_imgs_pil)} sample images from val set")

# -----
# 3. PANEL - 6-IMAGE PREDICTION RESULTS
# -----
fig = plt.figure(figsize=(22, 5 * len(sample_imgs_pil)))
fig.patch.set_facecolor('#f8f9fa')

for idx, (pil_img, true_lbl, fpath) in enumerate(
    zip(sample_imgs_pil, sample_true_lbls, sample_paths)):

    probs, pred_idx, pred_label, confidence, ood_flag = predict_single(pil_img)
    top5_idx = np.argsort(probs)[::-1][0:5]
    top5_probs = probs[top5_idx]
    top5_names = [CLASS_NAMES[i] for i in top5_idx]

    correct = (true_lbl == 'uploaded') or (true_lbl == CLASS_NAMES[pred_idx])
    border_col = C_GREEN if correct else C_RED
    if ood_flag:
        border_col = C_ORANGE

    # — Row layout: image | info card | confidence bars —
    gs_row = gridspec.GridSpec(
        1, 3,
        figure=fig,
        left=0.03, right=0.97,
        top=1 - idx/len(sample_imgs_pil) - 0.01,
        bottom=1 - (idx+1)/len(sample_imgs_pil) + 0.01,
        wspace=0.08
    )

```

```

# Column 0 – image
ax_img = fig.add_subplot(gs_row[0, 0])
tensor_for_display = infer_tfm(pil_img)
ax_img.imshow(denorm_display(tensor_for_display))
ax_img.set_xticks([]); ax_img.set_yticks([])
for spine in ax_img.spines.values():
    spine.set_edgecolor(border_col); spine.set_linewidth(3)
ax_img.set_title(f'Image {idx+1}', fontsize=10, fontweight='bold', color=

# Column 1 – info card
ax_info = fig.add_subplot(gs_row[0, 1])
ax_info.axis('off')
ax_info.set_facecolor('#ffffff')
verdict_col = C_ORANGE if ood_flag else (C_GREEN if correct else C_RED)
verdict_text = "NO SKIN ISSUE" if ood_flag else ("CORRECT" if correct else
info_lines = [
    ("Verdict",      verdict_text,      verdict_col,      14),
    ("Prediction",   short(pred_label, 38),      C_DARK,      11),
    ("Confidence",   f"{confidence*100:.1f}%",   C_BLUE,      12),
    ("True Label",   short(true_lbl, 38),      C_DARK,      10),
    ("OOD Flag",     str(ood_flag),           C_ORANGE if ood_flag else
    ("Top-2",        short(top5_names[1], 38),   '#555555',    9),
    ("Top-3",        short(top5_names[2], 38),   '#777777',    9),
]
y = 0.95
for label, val, col, fsize in info_lines:
    ax_info.text(0.05, y, f"{label}:", fontsize=9,
                 color='#666666', transform=ax_info.transAxes, va='top')
    ax_info.text(0.38, y, val, fontsize=fsize, fontweight='bold',
                 color=col, transform=ax_info.transAxes, va='top')
    y -= 0.135
for spine in ax_info.spines.values():
    spine.set_edgecolor('#dddddd'); spine.set_linewidth(1)

# Column 2 – top-5 confidence bars
ax_bar = fig.add_subplot(gs_row[0, 2])
bar_colors = [C_GREEN if i == 0 else C_BLUE for i in range(5)]
bars = ax_bar.barh(range(5), top5_probs * 100,
                   color=bar_colors, edgecolor='white', linewidth=0.6)
ax_bar.set_yticks(range(5))
ax_bar.set_yticklabels([short(n, 32) for n in top5_names], fontsize=8)
ax_bar.set_xlabel('Probability (%)', fontsize=9)
ax_bar.set_xlim(0, 105)
ax_bar.axvline(x=CONFIDENCE_THRESHOLD * 100, color='navy',
               linestyle='--', lw=1.5, label=f'Threshold {CONFIDENCE_THRE!
ax_bar.set_title('Top-5 Class Probabilities', fontsize=9, fontweight='bold
ax_bar.legend(fontsize=7, loc='lower right')
ax_bar.grid(axis='x', alpha=0.25)
for bar, val in zip(bars, top5_probs):
    ax_bar.text(val*100 + 0.5, bar.get_y() + bar.get_height()/2,

```

```

        f'{val*100:.1f}% ', va='center', fontsize=8, fontweight='b'
    for spine in ax_bar.spines.values():
        spine.set_edgecolor('#dddddd')

fig.suptitle('6-Image Prediction Panel - Skin Health Analyzer',
            fontsize=15, fontweight='bold', color=C_DARK, y=1.005)
plt.savefig('panel_6images.png', dpi=150, bbox_inches='tight',
            facecolor=fig.get_facecolor())

plt.show()
print("Saved: panel_6images.png")

# -----
# 4. FULL VALIDATION SET INFERENCE
# -----
print("\nRunning full validation inference ...")
all_preds, all_true, all_probs_list, all_conf, ood_flags = [], [], [], [], []

with torch.no_grad():
    for x, y in val_loader:
        x = x.cuda()
        logits = model(x)
        probs = F.softmax(logits, dim=1).cpu().numpy()
        preds = probs.argmax(axis=1)
        for i in range(len(y)):
            p = probs[i]
            conf = p.max()
            ent = -(p * np.log(p + 1e-9)).sum()
            ood = conf < CONFIDENCE_THRESHOLD or ent > np.log(NUM_CLASSES) *
            all_preds.append(preds[i])
            all_true.append(y[i].item())
            all_probs_list.append(p)
            all_conf.append(conf)
            ood_flags.append(ood)

all_preds = np.array(all_preds)
all_true = np.array(all_true)
all_probs = np.array(all_probs_list)
all_conf = np.array(all_conf)
ood_flags = np.array(ood_flags)

# -----
# 5. EVALUATION STATISTICS
# -----
acc = accuracy_score(all_true, all_preds)
f1_w = f1_score(all_true, all_preds, average='weighted', zero_division=0)
f1_mac = f1_score(all_true, all_preds, average='macro', zero_division=0)
f1_per_cls = f1_score(all_true, all_preds, average=None, zero_division=0)
prec_w = precision_score(all_true, all_preds, average='weighted', zero_division=0)
rec_w = recall_score(all_true, all_preds, average='weighted', zero_division=0)
top3_hits = np.array([int(all_true[i] in np.argsort(all_probs[i])[::-1][:3])
                      for i in range(len(all_true))])

```

```

top3_acc    = top3_hits.mean()
ood_rate    = ood_flags.mean()

per_cls_acc = []
per_cls_top3 = []
for i in range(NUM_CLASSES):
    m = all_true == i
    per_cls_acc.append(accuracy_score(all_true[m], all_preds[m]) if m.sum() > 0 else 0)
    per_cls_top3.append(top3_hits[m].mean() if m.sum() > 0 else 0)

print("\n" + "="*65)
print("  EVALUATION STATISTICS  -  Skin Health Analyzer")
print("="*65)
print(f"  Validation images          : {len(all_true)}")
print(f"  Overall Top-1 Accuracy      : {acc*100:.2f}%")
print(f"  Overall Top-3 Accuracy      : {top3_acc*100:.2f}%")
print(f"  Weighted F1                 : {f1_w:.4f}")
print(f"  Macro F1                    : {f1_mac:.4f}")
print(f"  Weighted Precision          : {prec_w:.4f}")
print(f"  Weighted Recall             : {rec_w:.4f}")
print(f"  Avg Confidence              : {all_conf.mean()*100:.1f}%")
print(f"  OOD / No Skin Issue Rate    : {ood_rate*100:.1f}%")
print("="*65)
print()
print(classification_report(all_true, all_preds,
                           target_names=CLASS_NAMES,
                           zero_division=0, digits=4))

stats_dict = {
    "top1_accuracy": round(float(acc), 4),
    "top3_accuracy": round(float(top3_acc), 4),
    "weighted_f1":    round(float(f1_w), 4),
    "macro_f1":       round(float(f1_mac), 4),
    "weighted_precision": round(float(prec_w), 4),
    "weighted_recall":  round(float(rec_w), 4),
    "avg_confidence": round(float(all_conf.mean()), 4),
    "ood_rate":        round(float(ood_rate), 4),
    "num_classes":     int(NUM_CLASSES),
    "val_images":      int(len(all_true)),
    "confidence_threshold": CONFIDENCE_THRESHOLD,
    "per_class": {
        CLASS_NAMES[i]: {
            "accuracy": round(float(per_cls_acc[i]), 4),
            "top3_accuracy": round(float(per_cls_top3[i]), 4),
            "f1": round(float(f1_per_cls[i]), 4)
        } for i in range(NUM_CLASSES)
    }
}

with open('evaluation_stats.json', 'w') as f:
    json.dump(stats_dict, f, indent=2)
print("Saved: evaluation_stats.json")

```

```

# -----
# 6. GRAPH 1 – Summary metrics bar chart
# -----

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
fig.patch.set_facecolor('#f8f9fa')
fig.suptitle('Model Performance Summary', fontsize=14, fontweight='bold', color=

# 6a. Global metrics
metric_names = ['Top-1 Acc', 'Top-3 Acc', 'Weighted F1',
                'Macro F1', 'Precision', 'Recall']
metric_values = [acc, top3_acc, f1_w, f1_mac, prec_w, rec_w]
bar_cols = [C_GREEN if v >= 0.75 else C_ORANGE if v >= 0.55 else C_RED
            for v in metric_values]
b = axes[0].bar(metric_names, metric_values, color=bar_cols, edgecolor='white')
axes[0].set_ylim(0, 1.12)
axes[0].set_ylabel('Score', fontsize=10)
axes[0].set_title('Global Metrics', fontsize=11, fontweight='bold')
axes[0].tick_params(axis='x', rotation=25, labelsiz=8)
axes[0].axhline(0.75, color=C_GREEN, linestyle='--', lw=1.2, alpha=0.5)
axes[0].axhline(0.55, color=C_RED, linestyle='--', lw=1.2, alpha=0.5)
for bar, val in zip(b, metric_values):
    axes[0].text(bar.get_x() + bar.get_width()/2, val + 0.02,
                 f'{val*100:.1f}%', ha='center', fontsize=9, fontweight='bold')
axes[0].grid(axis='y', alpha=0.3)

# 6b. Per-class accuracy sorted
sorted_idx = np.argsort(per_cls_acc)
sorted_acc = [per_cls_acc[i] for i in sorted_idx]
sorted_lbls = [short(CLASS_NAMES[i], 32) for i in sorted_idx]
colors_cls = [C_RED if a < 0.60 else C_ORANGE if a < 0.75 else C_GREEN
              for a in sorted_acc]
axes[1].barh(range(NUM_CLASSES), sorted_acc, color=colors_cls,
             edgecolor='white', linewidth=0.4)
axes[1].set_yticks(range(NUM_CLASSES))
axes[1].set_yticklabels(sorted_lbls, fontsize=7)
axes[1].set_xlabel('Accuracy', fontsize=10)
axes[1].set_title('Per-Class Accuracy\n(sorted low→high)', fontsize=11, fontw
axes[1].set_xlim(0, 1.12)
axes[1].axvline(0.75, color=C_GREEN, linestyle='--', lw=1, alpha=0.5)
axes[1].axvline(0.60, color=C_RED, linestyle='--', lw=1, alpha=0.5)
for i, val in enumerate(sorted_acc):
    axes[1].text(val + 0.01, i, f'{val*100:.0f}%', va='center', fontsize=6)
axes[1].grid(axis='x', alpha=0.3)

# 6c. Top-1 vs Top-3 per class
x = np.arange(NUM_CLASSES)
w = 0.38
axes[2].barh(x - w/2, per_cls_acc, w, label='Top-1', color=C_BLUE, alpha=0
axes[2].barh(x + w/2, per_cls_top3, w, label='Top-3', color=C_PURPLE, alpha=0
axes[2].set_yticks(x)

```

```

axes[2].set_yticklabels([short(c, 30) for c in CLASS_NAMES], fontsize=7)
axes[2].set_xlabel('Accuracy', fontsize=10)
axes[2].set_title('Top-1 vs Top-3 per Class', fontsize=11, fontweight='bold')
axes[2].legend(fontsize=9)
axes[2].set_xlim(0, 1.12)
axes[2].grid(axis='x', alpha=0.3)

plt.tight_layout()
plt.savefig('graph_metrics_summary.png', dpi=150, bbox_inches='tight',
          facecolor='#f8f9fa')
plt.show()
print("Saved: graph_metrics_summary.png")

# -----
# 7. GRAPH 2 – Per-class F1 + Confidence distribution
# -----

fig, axes = plt.subplots(1, 2, figsize=(18, 7))
fig.patch.set_facecolor('#f8f9fa')

# 7a. Per-class F1 horizontal bar
sorted_f1_idx = np.argsort(f1_per_cls)
sorted_f1     = f1_per_cls[sorted_f1_idx]
sorted_f1_lbls = [short(CLASS_NAMES[i], 32) for i in sorted_f1_idx]
f1_colors     = [C_RED if v < 0.55 else C_ORANGE if v < 0.72 else C_GREEN
                  for v in sorted_f1]
axes[0].barh(range(NUM_CLASSES), sorted_f1, color=f1_colors,
             edgcolor='white', linewidth=0.4)
axes[0].set_yticks(range(NUM_CLASSES))
axes[0].set_yticklabels(sorted_f1_lbls, fontsize=7)
axes[0].set_xlabel('F1 Score', fontsize=10)
axes[0].set_title('Per-Class F1 Score (sorted)\nRed<0.55 | Orange<0.72 | Green',
                  fontsize=10, fontweight='bold')
axes[0].set_xlim(0, 1.1)
axes[0].axvline(0.72, color=C_GREEN, linestyle='--', lw=1, alpha=0.5)
axes[0].axvline(0.55, color=C_RED,   linestyle='--', lw=1, alpha=0.5)
for i, val in enumerate(sorted_f1):
    axes[0].text(val + 0.01, i, f'{val:.2f}', va='center', fontsize=6)
axes[0].grid(axis='x', alpha=0.3)

# 7b. Confidence histogram – correct vs incorrect
correct_mask = all_preds == all_true
axes[1].hist(all_conf[correct_mask], bins=40, alpha=0.7, color=C_GREEN,
             label=f'Correct ({correct_mask.sum()})', density=True)
axes[1].hist(all_conf[~correct_mask], bins=40, alpha=0.7, color=C_RED,
             label=f'Incorrect ({(~correct_mask).sum()})', density=True)
axes[1].axvline(CONFIDENCE_THRESHOLD, color='navy', linestyle='--', lw=2,
               label=f'OOD threshold ({CONFIDENCE_THRESHOLD})')
axes[1].set_xlabel('Max Softmax Confidence', fontsize=11)
axes[1].set_ylabel('Density', fontsize=11)
axes[1].set_title('Confidence Distribution\nCorrect vs Incorrect Predictions',
                  fontsize=11, fontweight='bold')

```